

Test I Study Guide

Illya Starikov

February 15, 2016

1 Sorting Algorithms

1.1 Insertion-Sort

```
1 Insertion-Sort(A)
2   for j = 2 to A.length
3     key = A[j]
4     // Insert A[j] into the sorted sequence A[1..j - 1]
5     i = j - 1
6     while i > 0 and A[i] > key
7       A[i + 1] = A[i]
8       i = i - 1
9     A[i + 1] = key
```

1.2 Merge Sort

```
1 Merge-Sort(A, p, r)
2   if p < r
3     q =  $\lfloor (p + r) / 2 \rfloor$ 
4     Merge-Sort(A, p, q)
5     Merge-Sort(A, q + 1, r)
6     Merge(A, p, q, r)
7
8 Merge(A, p, q, r)
9    $n_1 = q - p + 1$ 
10   $n_2 = r - q$ 
11  let L[1... $n_1 + 1$ ] and R[1... $n_2$ ] be new arrays
12  for i = 1 to  $n_1$ 
13    L[i] = A[p + i - 1]
14  for j = 1 to  $n_2$ 
15    R[j] = A[q + j]
16  R[ $n_1 + 1$ ] =  $\infty$ 
17  R[ $n_2 + 1$ ] =  $\infty$ 
18  i = 1
19  j = 1
```

```

20     for k = p to r
21         if L[i] ≤ R[j]
22             A[k] = L[i]
23             i = i + 1
24         else A[k] = R[j]
25             j = j + 1

```

1.3 Heap Sort

```

1  Max-Heapify(A, i)
2      l = Left(i)
3      r = Right(i)
4      if l ≤ A.heap-size and A[l] > A[i]
5          largest = l
6      else largest = i
7      if r ≤ A.heap-size and A[r] > A[largest]
8          largest = r
9      if largest ≠ i
10         exchange A[i] with A[largest]
11         Max-Heapify(A-largest)
12
13 Heapsort(A)
14     Build-Max-Heap(A)
15     for i = A.length downto 2
16         exchange A[1] with A[i]
17         A.heap-size = A.heap-size - 1
18         Max-Heapify(A, 1)

```

1.4 Quicksort

```

1  Quicksort(A, p, r)
2      if p < r
3          q = Partition(A, p, r)
4          Quicksort(A, p, q - 1)
5          Quicksort(A, q + 1, r)
6
7  Partition(A, p, r)
8      x = A[r]
9      i = p - 1
10     for j = p to r - 1
11         if A[j] ≤ x
12             i = i + 1
13             exchange A[i] with A[j]
14     exchange A[i + 1] with A[r]
15     return i + 1

```

1.5 Counting Sort

Let A = input array, B = output array, n = size of array, k = maximum number.

```
1 countingSort(A, B, n, k)
2   let C[0...k] be the counting array
3   for i = 0 to k
4       C[i] = 0
5   for j = 1 to n
6       C[A[j]] = C[A[j]] + 1
7   for i = 1 to k
8       C[i] = C[i] + C[i - 1]
9   for j = n down to 1
10      B[C[A[j]]] = A[j]
11      C[A[j]] = C[A[j]] - 1
```

1.6 Rate of Growth

Insertion Sort Worst: $\Theta(n^2)$, Average: $\Theta(n^2)$

Merge Sort Worst: $\Theta(n \lg n)$, Average: $\Theta(n \lg n)$

Quick Sort Worst: $\Theta(n^2)$, Average: $\Theta(n \lg n)$

Heap Sort Worst: $\Theta(n \lg n)$, Average: $\Theta(n \lg n)$

2 Growth Classes

2.1 O -notation

$O(g(n)) = \{f(n) : \text{there exists positive constant } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c g(n) \text{ for all } n \geq n_0\}$

2.2 Ω -notation

$\Omega(g(n)) = \{f(n) : \text{there exists positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c g(n) \leq f(n) \text{ for all } n \geq n_0\}$

2.3 Θ -notation

$\Theta(g(n)) = \{f(n) : \text{there exists positive constants } c_1, c_2 \text{ and } n_0 \text{ such that}$
 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\}$

2.4 o -notation (Little-o)

$f(n) \in o(g(n))$ iff for every $c > 0$ there is an $n_0 > 0$ such that

$$0 \leq f(n) < c g(n)$$

for all $n \geq n_0$

2.5 ω -notation (Little-omega)

$$f(n) \in \omega(g(n)) \text{ iff } g(n) \in o(f(n)).$$

or $f(n) \in \omega(g(n))$ iff for every $c > 0$ there is an $n_0 > 0$ such that

$$0 \leq c g(n) < f(n)$$

for all $n \geq n_0$

3 Master Theorem

The master theorem can only be applied to recurrence equations of the form:

$$T(n) = aT(n/b) + f(n)$$

3.1 Constants

n The size of the problem

a The number of subproblems

n/b The size of each subproblem

f(n) cost outside of recursive calls (divide, combine)

3.2 Cases

$f(n) \in O(n^{\log_b a - \epsilon})$	$T(n) \in \Theta(n^{\log_b a})$
$f(n) \in \Theta(n^{\log_b a})$	$T(n) \in \Theta(n^{\log_b a} \lg n)$
$f(n) \in \Omega(n^{\log_b a + \epsilon})$	$T(n) \in \Theta(f(n))$

4 Definitions

Algorithm Any well-defined computational procedure that takes a set of values as input and produces a set of values as output in a finite number of steps

Correct Algorithm One returns the correct solution for every valid instance of a problem

Loop Invariance Define a key property about the relationship among variables of the algorithms.

- Holds in the *initial* case.

Initialization The loop invariance must be true prior to the first iteration.

- Is *maintained* each iteration.

Maintenance If the property holds prior to an iteration, it must still hold after the iteration is complete.

- Yields correctness when the loop *terminates*.

Termination The invariant provides a useful property that helps demonstrate the algorithm is correct.

Heapify Go all the way down to the heap and fix the violations of the max-heap property by sifting-up